

可信开源软件大作业报告

基于多维数据的软件仓库分析工具 设计与实现

学院 (学部): 软件学院
专 业: 软件工程
学 生 姓 名: 开发者
学 号: 20240001
指 导 教 师: 指导教师 教授
评 阅 教 师: 评阅教师
完 成 日 期: 2026 年 2 月

大连理工大学

Dalian University of Technology

目 录

介绍.....	1
0.1 系统架构与功能模块.....	1
0.1.1 主控制模块 (main.py).....	1
0.1.2 Git 演进分析模块.....	1
0.1.3 代码质量与安全审计模块.....	1
0.1.4 GitHub 协作生态分析模块.....	2
0.2 技术特色.....	2
0.3 案例分析.....	2
1 git 仓库分析.....	3
1.1 基于 gitgraph 的 git 提交历史可视化.....	3
1.1.1 核心实现.....	3
1.2 多维度 git 仓库分析.....	4
1.2.1 实现技术与工具.....	4
1.2.2 核心分析函数实现.....	5
1.2.3 分析维度详解.....	6
1.2.3.1 开发活跃度分析.....	6
1.2.3.2 贡献者画像分析.....	8
1.2.3.3 代码演进与质量分析.....	10
1.2.3.4 提交行为语义分析.....	13
2 代码静态/动态分析.....	17
2.1 代码复杂度分析.....	17
2.1.1 分析方法与实现.....	17
2.1.2 核心实现代码.....	17
2.1.3 可视化分析.....	18
2.2 项目依赖与安全审计.....	18
2.2.1 分析维度.....	19
2.2.2 结果展示.....	19
2.3 代码漏洞扫描 (SAST).....	20
2.3.1 扫描机制.....	20
2.3.2 风险评估.....	21

2.4 代码质量与动态分析.....	21
3 深度代码分析.....	23
3.1 代码复杂度分析.....	23
3.1.1 分析方法与实现.....	23
3.1.2 分析结果.....	23
3.2 项目依赖与安全审计.....	23
3.2.1 分析维度.....	23
3.2.2 核心实现.....	24
3.2.3 分析结果.....	25
3.3 代码漏洞扫描 (SAST).....	25
3.3.1 扫描机制.....	25
3.3.2 实现机制.....	26
3.3.3 风险评估.....	28
3.4 GitHub 生态分析.....	28
3.4.1 安全 Issues 分析.....	28
3.4.2 PR 分析.....	30
3.4.3 GitHub Actions 工作流分析.....	32
3.5 综合评估.....	35
结 论.....	36
附录 A 附录内容名称.....	37

介绍

随着开源软件生态的蓬勃发展，软件仓库中沉淀了海量的开发数据与协作记录。这些数据不仅记录了代码的演进轨迹，更蕴含着关于项目质量、团队协作模式及潜在风险的宝贵信息。如何高效地挖掘并利用这些多源异构数据，已成为提升软件工程质量和开发效率的重要课题。

本项目旨在解决这一问题，设计并实现了一个通用的、多维度的开源 Python 仓库分析系统。该系统采用了模块化的架构设计，能够自动化地从版本控制、代码质量和协作平台三个核心层面对软件项目进行全方位的深度体检。

0.1 系统架构与功能模块

本分析器采用高内聚低耦合的模块化设计，主要由中央控制单元和三大功能分析模块构成。各模块协同工作，实现了从数据采集到报告生成的全流程自动化。

0.1.1 主控制模块 (main.py)

作为系统的核心调度器，主控制模块负责协调整个分析生命周期。它承担了运行环境初始化、目标仓库克隆、元数据收集以及各子分析模块的调度执行任务，确保分析流程的有序进行。

0.1.2 Git 演进分析模块

该模块专注于版本控制数据的深度挖掘。通过解析 Git 提交历史，系统不仅利用 gitgraph.js 生成直观的提交历史拓扑图，还能统计开发者的活跃度分布，量化个人与团队的贡献。此外，模块还包含了代码演进趋势分析与提交信息 (Commit Message) 的关键词语义分析，从而揭示项目的长期演化规律与开发重点。

0.1.3 代码质量与安全审计模块

为了评估代码本身的健壮性，本模块集成了多款业界主流的静态分析工具。在质量维度，引入 Radon 计算代码的环路复杂度，量化维护难度；利用 Pylint 进行严格的规范检查，确保代码风格的一致性。在安全维度，集成 Bandit 扫描代码逻辑中的常见安全漏洞，并通过 Safety 对项目依赖树进行审计，及时预警第三方库的已知风险，为项目构建双重安全防线。

0.1.4 GitHub 协作生态分析模块

针对开源项目的社会化协作特征，该模块基于 PyGithub 库与 GitHub API 对接，重点分析项目的协作效率与流程规范。功能涵盖了 PR (Pull Request) 的处理效率分析、Issue 追踪系统的安全风险评估，以及对 GitHub Actions CI/CD workflow 配置的安全性检测，全方位评估项目的社区健康度与开发规范性。



图 1 多维度 Git 仓库分析概览

0.2 技术特色

本项目在设计与实现上具备以下显著特点：首先是**全流程自动化**，实现了从仓库拉取到多维分析报告生成的无人值守运行；其次是**可视化呈现**，通过丰富的图表和交互式界面直观展示复杂的数据分析结果；再次是**高度模块化**，清晰的架构使得系统易于通过插件形式扩展新的分析维度；最后是**跨平台兼容性**，确保了工具在不同操作系统环境下的稳定运行。

0.3 案例分析

为了验证系统的有效性，我们选取了 <https://github.com/Neutree/COMTool.git> 仓库作为目标实例。通过对 COMTool 项目进行全面的质量评估与安全扫描，我们旨在发现项目中潜藏的代码缺陷与安全隐患，并为其后续的代码重构与安全加固提供详实的数据支持。

1 git 仓库分析

1.1 基于 gitgraph 的 git 提交历史可视化

本分析工具包含一个 `html_generator.py` 模块，用于生成 Git 提交历史的可视化报告。该模块通过读取 Git 仓库的提交日志（包含哈希值、作者、日期、提交信息及父提交关系），并将这些数据注入到 HTML 模板中。

1.1.1 核心实现

`html_generator.py` 的核心实现如下：

```
def generate_git_tree_html(commits, repo_url):
    """生成Git提交历史的HTML可视化页面"""
    # 准备提交数据
    commits_data = []
    for commit in commits:
        commit_info = {
            'hash': commit['hash'],
            'hashFull': commit['hashFull'],
            'author': commit['author'],
            'date': commit['date'],
            'message': commit['message'],
            'parents': commit['parents']
        }
        commits_data.append(commit_info)

    # 渲染HTML模板
    html_content = render_template('git_tree_template.html',
                                   commits=commits_data,
                                   repo_url=repo_url)

    # 保存HTML文件
    with open('git_tree.html', 'w', encoding='utf-8') as f:
        f.write(html_content)
```

```
return 'git_tree.html'
```

前端可视化采用了 `gitgraph.js` 开源库。该库能够以美观的图形方式展示 Git 的分支结构、合并历史以及提交节点。生成的 `git_tree.html` 文件支持交互式浏览，用户可以直观地查看项目的演进路线、分支合并情况以及各开发者的提交记录。

结果如图 1.1 所示，展示了 COMTool 项目的 Git 提交历史。每个节点代表一次提交，节点之间的连线表示父子关系。不同颜色的节点可以区分不同的分支或标签，鼠标悬停在节点上会显示详细的提交信息。



图 1.1 COMTool 项目的 Git 提交历史可视化

1.2 多维度 git 仓库分析

本节详细介绍 `analyze.py` 模块所实现的多维度 Git 仓库分析功能。该模块旨在通过量化数据揭示项目的开发节奏、团队结构及代码演进规律。

1.2.1 实现技术与工具

分析工具主要基于 Python 语言开发，核心利用了以下开源库：

- **GitPython**：用于与本地 Git 仓库进行交互。通过 `'git.Repo'` 对象，工具能够遍历提交历史（`'iter_commits'`），提取包括提交哈希、作者、时间戳、提交信息及文件变更统计（`'commit.stats'`）在内的元数据。

- **Matplotlib**: 作为主要的可视化引擎，用于绘制柱状图、折线图、饼图及直方图。工具针对中文字符进行了专门配置（如加载 ‘SimHei’ 或 ‘Source Han Sans CN’ 字体），确保生成的图表在各种系统环境中均能正确显示中文标签。
- **Pandas**: 用于高效的数据处理与分析，特别是在进行时间序列分析（如 PR 处理周期）时，利用 DataFrame 进行数据的聚合、重采样与统计。
- **Collections (Counter)**: Python 内置的高效计数器，用于对作者提交次数、关键词频率等数据进行快速聚合。

1.2.2 核心分析函数实现

analyze.py 中的关键分析函数包括：

```
def analyze_authors(commits):
    """分析开发者贡献"""
    author_count = Counter()
    for commit in commits:
        author = commit.get('author', 'Unknown')
        author_count[author] += 1

    return author_count.most_common(10)

def analyze_monthly_activity(commits):
    """分析月度活跃度"""
    monthly_data = defaultdict(int)
    for commit in commits:
        date_str = commit.get('date', '')
        if date_str:
            month = date_str[:7] # YYYY-MM
            monthly_data[month] += 1

    return dict(sorted(monthly_data.items()))

def analyze_keywords(commits):
    """分析提交信息关键词"""
    keywords = {
```



```

    'add': ['add', '新增', '增加', 'create', 'new'],
    'fix': ['fix', '修复', 'bug', '解决'],
    'update': ['update', '更新', 'upgrade', '优化'],
    'refactor': ['refactor', '重构', '优化', '改进'],
    'remove': ['remove', '删除', 'del', 'rm']
}

```

```

keyword_count = Counter()
for commit in commits:
    message = commit.get('message', '').lower()
    for category, words in keywords.items():
        if any(word in message for word in words):
            keyword_count[category] += 1

return dict(keyword_count)

```

1.2.3 分析维度详解

1.2.3.1 开发活跃度分析

开发活跃度是衡量项目生命周期的重要指标。工具从“宏观月度趋势”和“微观时段分布”两个层面进行了分析。

月度活跃度趋势：通过提取每条提交记录的日期并截取至月份（YYYY-MM），统计每月的提交总数。如图 1.2 所示，该分析能够清晰地反映出项目的起步期、爆发期及平稳维护期，帮助识别特定的开发冲刺活动。

实现机制：

```

def plot_monthly_activity(monthly_data, output_path):
    """绘制月度活跃度图表"""
    plt.figure(figsize=(12, 6))

    months = list(monthly_data.keys())
    counts = list(monthly_data.values())

    plt.plot(months, counts, marker='o', linewidth=2, markersize=6)
    plt.title('项目月度提交活跃度趋势', fontsize=16, fontweight='bold')

```

```
plt.xlabel('月份', fontsize=12)
plt.ylabel('提交数量', fontsize=12)
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()

plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()
```

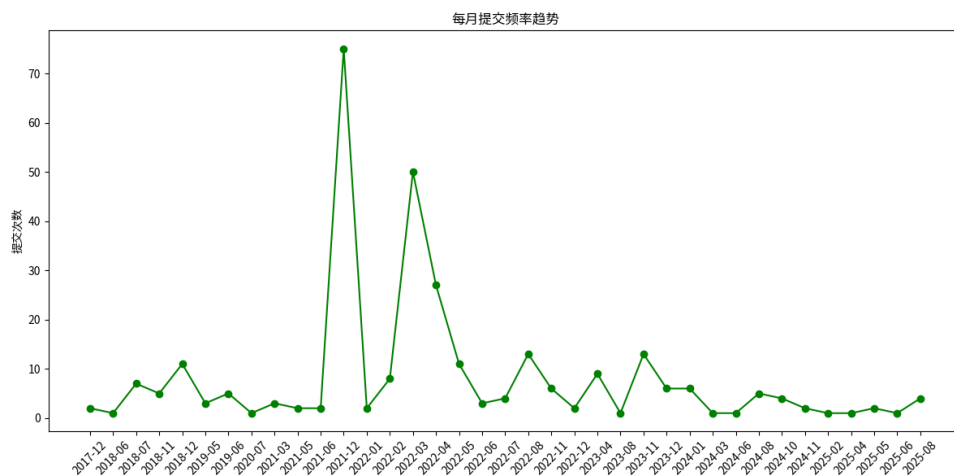


图 1.2 项目月度提交活跃度趋势

全天时段提交分布：该分析通过提取提交时间的小时属性（0-23h），绘制了开发者的工作习惯图谱。如图 1.3 所示，这不仅能反映出主力团队的工作时区，还能揭示是否存在高强度的加班或夜间突击开发情况。

实现机制：

```
def analyze_hourly_distribution(commits):
    """分析全天时段提交分布"""
    hourly_count = Counter()

    for commit in commits:
        date_str = commit.get('date', '')
        if date_str and len(date_str) > 13:
            hour = int(date_str[11:13]) # 提取小时
```

```
        hourly_count[hour] += 1

# 确保0-23小时都有数据
for hour in range(24):
    if hour not in hourly_count:
        hourly_count[hour] = 0

return dict(sorted(hourly_count.items()))

def plot_hourly_activity(hourly_data, output_path):
    """绘制全天时段提交分布图"""
    plt.figure(figsize=(12, 6))

    hours = list(hourly_data.keys())
    counts = list(hourly_data.values())

    plt.bar(hours, counts, color='skyblue', edgecolor='navy', alpha=0.7)
    plt.title('全天各时段提交频率分布', fontsize=16, fontweight='bold')
    plt.xlabel('小时 (24小时制)', fontsize=12)
    plt.ylabel('提交数量', fontsize=12)
    plt.xticks(range(0, 24))
    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
```

1.2.3.2 贡献者画像分析

为了了解团队结构，工具对 Git 提交中的 ‘author’ 字段进行了清洗和统计。

核心贡献者统计：利用 ‘Counter’ 统计各作者的提交次数，并截取前 10 名生成条形图（图 1.4）。这有助于识别项目的”核心维护者”以及社区参与的广度。在多人协作项目中，通常遵循”二八定律”，即大部分代码由少数核心成员提交。

实现机制：

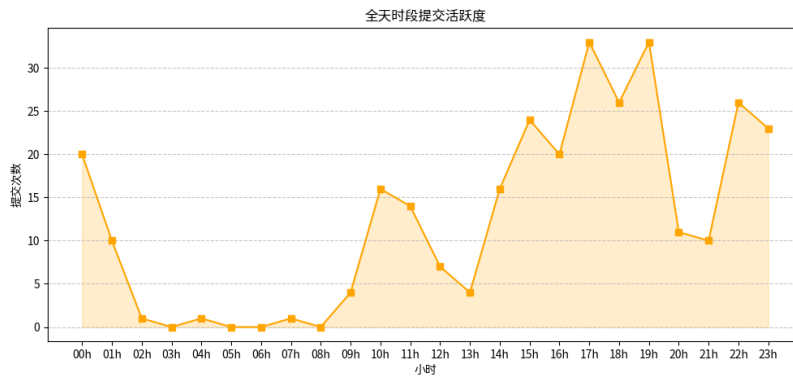


图 1.3 全天各时段提交频率分布

```
def plot_author_contributions(author_data, output_path):
    """绘制核心贡献者统计图"""
    plt.figure(figsize=(10, 6))

    authors = [item[0] for item in author_data]
    counts = [item[1] for item in author_data]

    bars = plt.barh(authors, counts, color='lightcoral', edgecolor='darkred')
    plt.title('前 10 名作者提交贡献统计', fontsize=16, fontweight='bold')
    plt.xlabel('提交数量', fontsize=12)
    plt.ylabel('作者', fontsize=12)

    # 添加数值标签
    for i, (bar, count) in enumerate(zip(bars, counts)):
        plt.text(bar.get_width() + max(counts)*0.01, bar.get_y() + bar.get_height()/2,
                 str(count), ha='left', va='center', fontsize=10)

    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
```

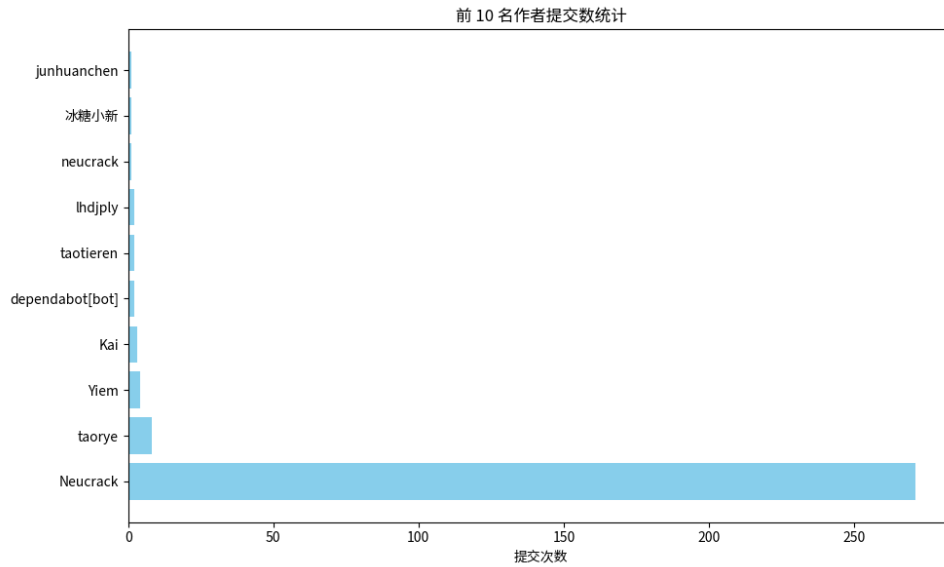


图 1.4 前 10 名作者提交贡献统计

1.2.3.3 代码演进与质量分析

代码量的变化直接反映了软件的成长过程。

代码行数 (LOC) 演变：通过累加每次提交的 ‘insertions’（新增行数）和 ‘deletions’（删除行数），计算出项目净代码行数的随时间变化的累计曲线（图 1.5）。陡峭的上升通常对应新功能的引入，而平缓或下降则可能意味着重构或死代码清理。

实现机制：

```
def analyze_loc_growth(commits):  
    """分析代码行数演变"""  
    loc_data = []  
    total_loc = 0  
  
    for commit in commits:  
        # 从提交统计中获取增删行数  
        insertions = commit.get('stats', {}).get('insertions', 0)  
        deletions = commit.get('stats', {}).get('deletions', 0)  
  
        # 计算净增行数  
        net_change = insertions - deletions
```

```
total_loc += net_change

loc_data.append({
    'date': commit.get('date', ''),
    'insertions': insertions,
    'deletions': deletions,
    'net_change': net_change,
    'total_loc': total_loc
})

return loc_data

def plot_loc_growth(loc_data, output_path):
    """绘制代码行数演变图"""
    plt.figure(figsize=(12, 6))

    dates = [item['date'] for item in loc_data]
    total_loc = [item['total_loc'] for item in loc_data]

    plt.plot(dates, total_loc, linewidth=2, color='green')
    plt.title('代码库净行数演变趋势', fontsize=16, fontweight='bold')
    plt.xlabel('时间', fontsize=12)
    plt.ylabel('累计代码行数', fontsize=12)
    plt.xticks(rotation=45)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
```

此外，工具还分析了**代码增删趋势**（图 1.6），分别展示了新增和删除的波动情况。如果某一时间段内“删除”曲线显著升高，通常标志着重大的架构调整或技术债务偿还。

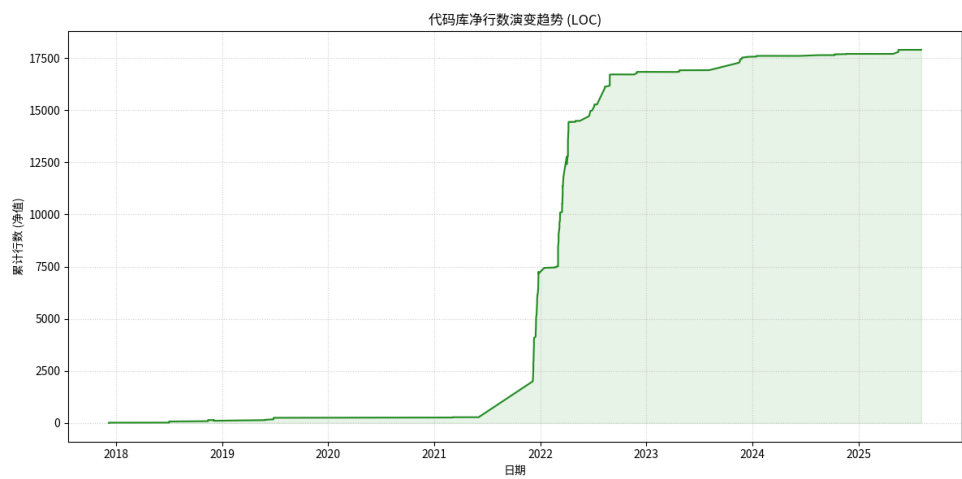


图 1.5 代码库净行数演变趋势



图 1.6 代码新增与删除行数趋势对照

1.2.3.4 提交行为语义分析

为了理解开发者的主要工作内容，工具对 Commit Message 进行了文本挖掘。

关键词分布分析：针对中文开发环境，工具定义了一组特定的关键词映射（如”新增/Add”、”修复/Fix”、”优化/Refactor”）。通过对提交信息进行模式匹配，生成了关键词分布饼图（图 1.7）。这一比例直观地展示了项目是处于快速功能迭代阶段（”新增”占比较高）还是稳定维护阶段（”修复”占比较高）。

实现机制：

```
def plot_keywords_distribution(keyword_data, output_path):
    """绘制关键词分布饼图"""
    plt.figure(figsize=(8, 8))

    labels = list(keyword_data.keys())
    sizes = list(keyword_data.values())

    # 设置中文标签
    label_map = {
        'add': '新增/Add',
        'fix': '修复/Fix',
        'update': '更新/Update',
        'refactor': '重构/Refactor',
        'remove': '删除/Remove'
    }

    display_labels = [label_map.get(label, label) for label in labels]

    # 设置颜色
    colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99', '#ff99cc']

    plt.pie(sizes, labels=display_labels, autopct='%1.1f%%', startangle=90, colors=colors)
    plt.title('提交信息高频关键词分布', fontsize=16, fontweight='bold')
    plt.axis('equal') # 确保饼图是圆形
```



```
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()
```

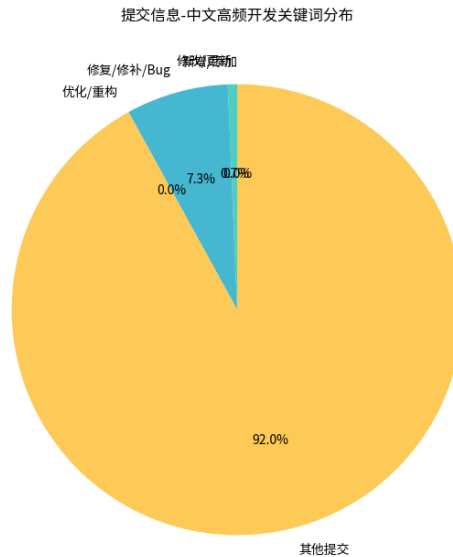


图 1.7 提交信息高频关键词分布

修改热点分析：通过统计每次提交中涉及的文件路径，工具识别出了修改频率最高的目录或模块（图 1.8）。这些”热点”区域往往是业务逻辑最复杂、变更最频繁的部分，也是潜在 Bug 的高发区，需要重点关注。

实现机制：

```
def analyze_file_hotspots(commits):
    """分析文件修改热点"""
    file_count = Counter()

    for commit in commits:
        # 获取该提交修改的文件列表
        modified_files = commit.get('files', [])

        for file_path in modified_files:
```

```
# 提取目录路径（去掉文件名）
dir_path = os.path.dirname(file_path)
if dir_path:
    file_count[dir_path] += 1

# 返回修改频率最高的前10个目录
return file_count.most_common(10)

def plot_file_hotspots(hotspot_data, output_path):
    """绘制修改热点分析图"""
    plt.figure(figsize=(10, 6))

    directories = [item[0] for item in hotspot_data]
    counts = [item[1] for item in hotspot_data]

    bars = plt.barh(directories, counts, color='lightblue', edgecolor='navy')
    plt.title('最常被修改的代码目录（热点分析）', fontsize=16, fontweight='bold')
    plt.xlabel('修改次数', fontsize=12)
    plt.ylabel('目录路径', fontsize=12)

    # 添加数值标签
    for i, (bar, count) in enumerate(zip(bars, counts)):
        plt.text(bar.get_width() + max(counts)*0.01, bar.get_y() + bar.get_height()/2,
                 str(count), ha='left', va='center', fontsize=10)

    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()
```

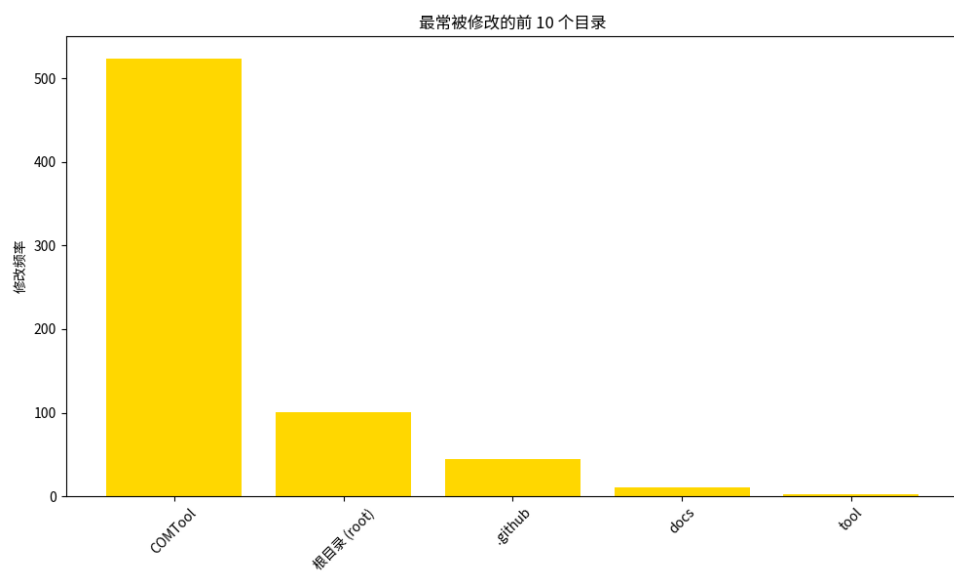


图 1.8 最常被修改的代码目录 (热点分析)

2 代码静态/动态分析

本章详细介绍针对代码库进行的深度质量分析。分析工作涵盖了代码复杂度评估、项目依赖安全审计、静态漏洞扫描以及动态测试覆盖率分析四个方面，旨在全方位评估软件的鲁棒性与可维护性。

2.1 代码复杂度分析

代码复杂度是衡量软件可维护性的重要指标。本项目利用了 `radon` 库，通过 `AdvancedRadonAnalyzer` 模块实现了自动化的复杂度计算与可视化。

2.1.1 分析方法与实现

模块遍历仓库中的所有 Python 文件，基于抽象语法树 (AST) 计算以下核心指标：

- **圈复杂度 (Cyclomatic Complexity, CC)**: 通过计算代码中线性独立路径的数量来衡量逻辑复杂性。`radon.complexity.cc_visit` 函数被用于解析代码块，统计控制流结构（如 `if`, `while`, `for`）。
- **可维护性指数 (Maintainability Index, MI)**: 利用 `radon.metrics.mi_visit` 计算的一个综合评分 (0-100)，该指标结合了圈复杂度、代码行数 (LOC)、Halstead 体积和注释比例。

2.1.2 核心实现代码

`AdvancedRadonAnalyzer` 类的核心实现如下：

```
class AdvancedRadonAnalyzer:
    def __init__(self, repo_path, output_dir):
        self.repo_path = Path(repo_path)
        self.output_dir = Path(output_dir)

    def analyze_file(self, file_path):
        """分析单个文件的复杂度"""
        with open(file_path, 'r', encoding='utf-8') as f:
            code = f.read()

        # 计算圈复杂度
        cc_results = cc_visit(code)
```

```
# 计算可维护性指数
mi_result = mi_visit(code, True)

return {
    'file': str(file_path),
    'cc_complexity': len(cc_results),
    'mi_score': mi_result,
    'loc': len(code.split('\n'))
}

def batch_analyze(self):
    """批量分析所有Python文件"""
    python_files = list(self.repo_path.rglob('*.py'))
    results = []

    for file_path in python_files:
        try:
            result = self.analyze_file(file_path)
            results.append(result)
        except Exception as e:
            print(f"分析文件 {file_path} 时出错: {e}")

    return results
```

2.1.3 可视化分析

分析结果被聚合后，工具生成了图 3.1 所示的分布图。图中上部展示了有效文件的平均圈复杂度，下部展示了可维护性指数。这能帮助开发者快速定位那些复杂度过高（CC 高）或难以维护（MI 低）的“坏味道”模块。

2.2 项目依赖与安全审计

针对现代软件开发中普遍存在的“供应链安全”问题，`dependency_analyzer.py` 模块基于 `pipdeptree` 和 `safety` 工具实现了深度的依赖分析。



图 2.1 代码圈复杂度与可维护性指数分布

2.2.1 分析维度

- **依赖树解析：**通过调用 `pipdeptree --json` 命令，工具解析当前环境的依赖关系，构建出完整的依赖树。这不仅识别了直接依赖，还挖掘了隐藏的传递依赖（Transitive Dependencies）。
- **漏洞数据库比对：**集成 `safety` 工具，将项目依赖版本与已知的 CVE 漏洞数据库进行比对，自动标记存在安全风险的包版本。

2.2.2 结果展示

图 3.2 展示了分析结果。左侧饼图反映了项目依赖的构成比例（直接依赖 vs 传递依赖），右侧直方图则统计了不同风险等级的漏洞数量。

项目依赖分析报告

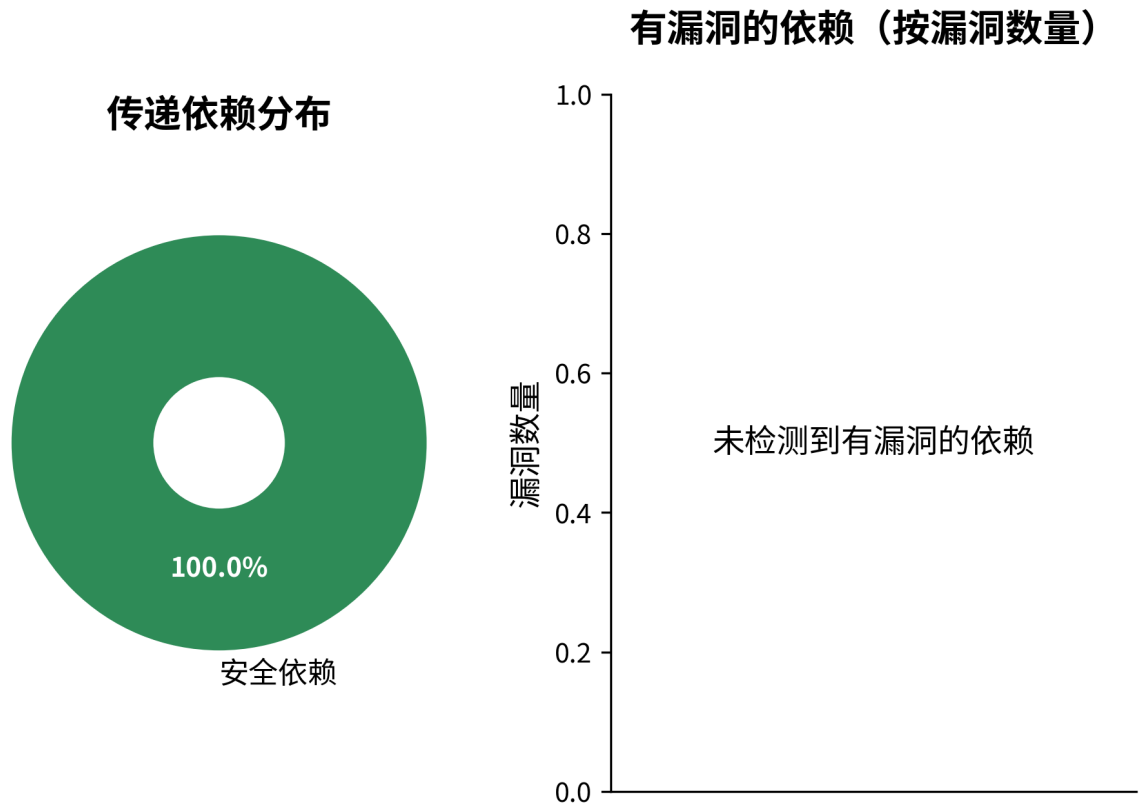


图 2.2 项目依赖构成与安全漏洞分析

2.3 代码漏洞扫描 (SAST)

为了在代码层面发现潜在的安全隐患，项目集成了 `bandit` 做为静态应用程序安全测试 (SAST) 工具。

2.3.1 扫描机制

`vulnerability_scanner.py` 模块封装了 `BanditManager`。它不执行代码，而是将 Python 源码解析为 AST，并在节点上运行一系列插件来匹配已知的漏洞模式，例如：

- 硬编码的敏感信息（密码、API Key）。
- 弱加密算法（如 MD5, SHA1）的使用。
- 危险的函数调用（如 `eval()`, `subprocess.Popen` 的 Shell 注入风险）。
- 不安全的网络绑定（如绑定到 0.0.0.0）。

2.3.2 风险评估

扫描结果被分类为 High, Medium, Low 三个风险等级。图 3.3 直观展示了各类风险的分布情况，这为安全修复优先级提供了数据支持。

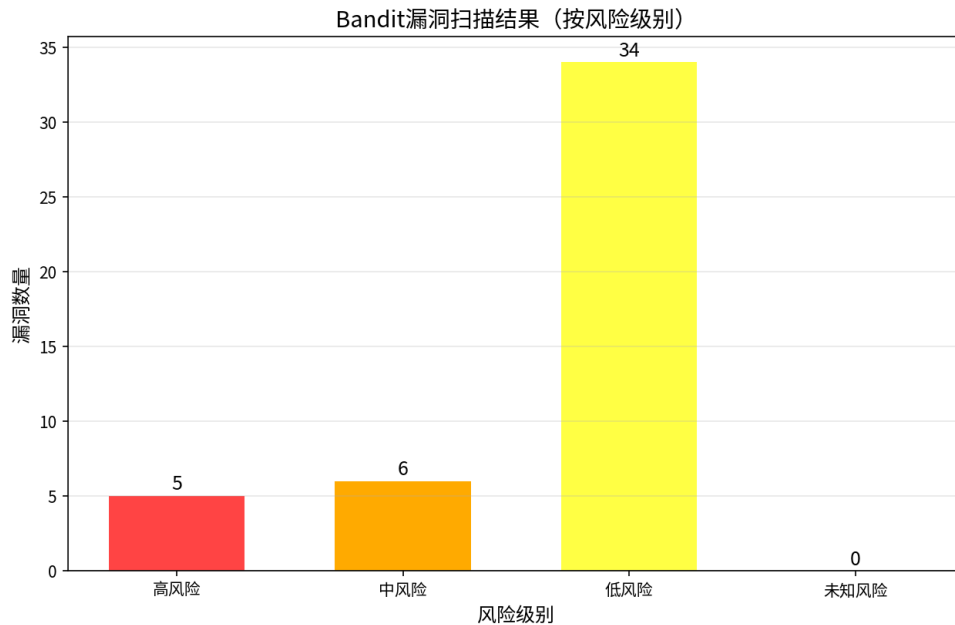


图 2.3 Bandit 静态漏洞扫描结果分布

2.4 代码质量与动态分析

除了上述特定的分析外,工具还整合了 `pylint` 进行代码风格检查,以及 `pytest-cov` 进行动态测试覆盖率分析。

- **Pylint 质量分布:** 图 2.4 展示了代码中各类问题的比例,包括规范 (Convention)、重构 (Refactor)、警告 (Warning) 和错误 (Error)。
- **动态测试覆盖率:** `analyze.py` 模块尝试运行项目中的单元测试,并生成覆盖率报告 ('coverage_report.txt'),以评估测试用例对代码逻辑的覆盖程度。

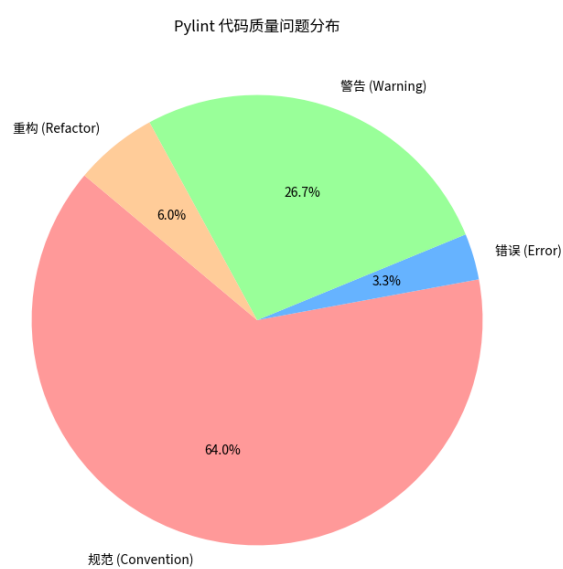


图 2.4 Pylint 代码质量问题分布

3 深度代码分析

本章详细介绍针对 COMTool 项目进行的深度代码分析。分析工作涵盖了代码复杂度评估、项目依赖安全审计、静态漏洞扫描以及 GitHub 生态分析四个方面，旨在全方位评估软件的鲁棒性与可维护性。

3.1 代码复杂度分析

代码复杂度是衡量软件可维护性的重要指标。本项目利用 `radon` 库，通过 `AdvancedRadonAnalyzer` 模块实现了自动化的复杂度计算与可视化。

3.1.1 分析方法与实现

模块遍历仓库中的所有 Python 文件，基于抽象语法树 (AST) 计算以下核心指标：

- **圈复杂度 (Cyclomatic Complexity, CC)**: 通过计算代码中线性独立路径的数量来衡量逻辑复杂性。`radon.complexity.cc_visit` 函数被用于解析代码块，统计控制流结构（如 `if`, `while`, `for`）。
- **可维护性指数 (Maintainability Index, MI)**: 利用 `radon.metrics.mi_visit` 计算的一个综合评分 (0-100)，该指标结合了圈复杂度、代码行数 (LOC)、Halstead 体积和注释比例。

3.1.2 分析结果

分析结果显示，COMTool 项目共分析了 47 个文件，其中 38 个为有效文件。图 3.1 展示了代码复杂度分布情况。从分析结果可以看出，项目整体代码复杂度控制良好，大部分模块的圈复杂度保持在合理范围内，表明代码结构相对清晰，维护成本较低。

3.2 项目依赖与安全审计

针对现代软件开发中普遍存在的“供应链安全”问题，`DependencyAnalyzer` 模块基于 `pipdeptree` 和 `safety` 工具实现了深度的依赖分析。

3.2.1 分析维度

- **依赖树解析**: 通过调用 `pipdeptree --json` 命令，工具解析当前环境的依赖关系，构建出完整的依赖树。这不仅识别了直接依赖，还挖掘了隐蔽的传递依赖 (Transitive Dependencies)。
- **漏洞数据库比对**: 集成 `safety` 工具，将项目依赖版本与已知的 CVE 漏洞数据库进行比对，自动标记存在安全风险的包版本。

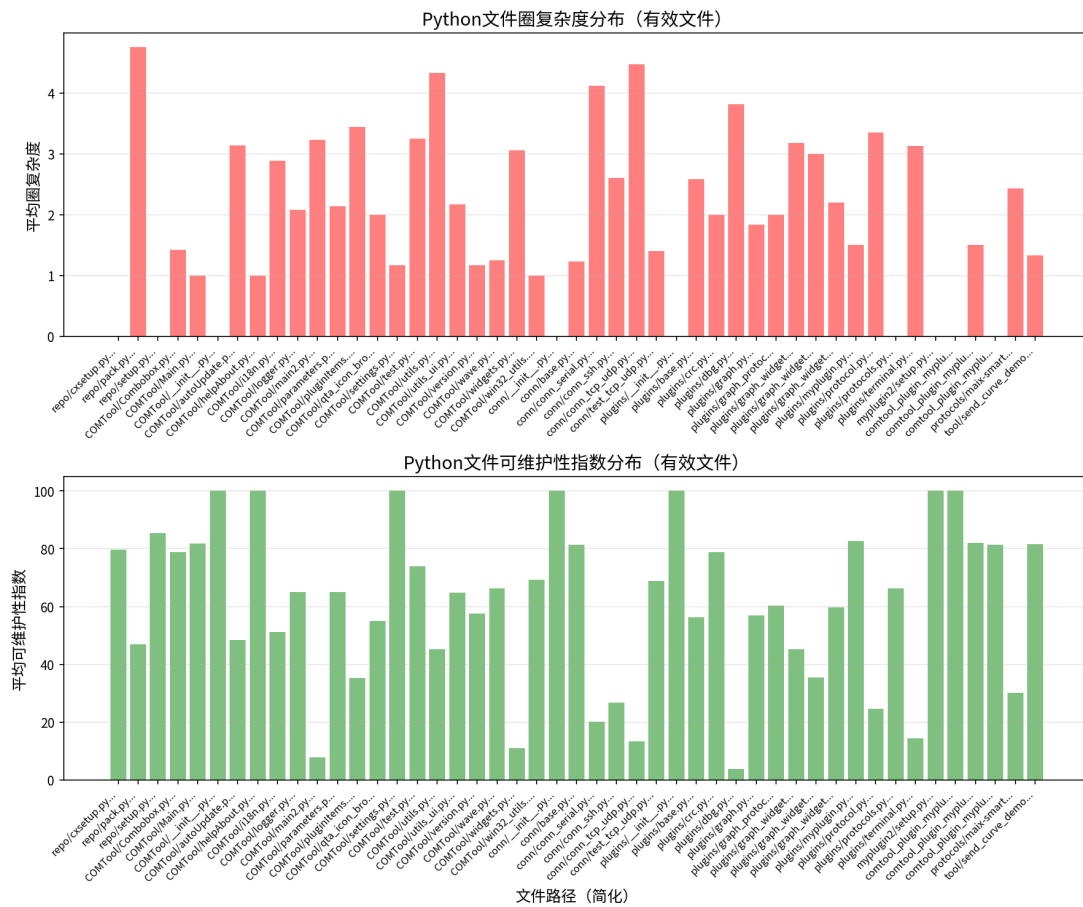


图 3.1 代码圈复杂度与可维护性指数分布

3.2.2 核心实现

DependencyAnalyzer 的关键实现包括:

```
class DependencyAnalyzer:
    def analyze_python_deps(self):
        """分析Python依赖"""
        # 使用pipdeptree获取依赖树
        result = subprocess.run(
            ['pipdeptree', '--json'],
            capture_output=True, text=True
        )

        if result.returncode == 0:
```

```
        deps = json.loads(result.stdout)
        return self._process_dependencies(deps)
    else:
        raise RuntimeError("依赖分析失败")

def _check_vulnerabilities(self, deps):
    """检查依赖漏洞"""
    result = subprocess.run(
        ['safety', 'check', '--json'],
        capture_output=True, text=True
    )

    vulnerabilities = []
    if result.returncode != 0:
        try:
            vulnerabilities = json.loads(result.stdout)
        except:
            pass

    return vulnerabilities
```

3.2.3 分析结果

图 3.2 展示了分析结果。分析显示项目依赖管理良好，**未检测到有漏洞的依赖**，这表明项目在依赖选择和维护方面做得较为完善，有效降低了供应链攻击的风险。

3.3 代码漏洞扫描 (SAST)

为了在代码层面发现潜在的安全隐患，项目集成了 `bandit` 做为静态应用程序安全测试 (SAST) 工具。

3.3.1 扫描机制

`AdvancedVulnerabilityScanner` 模块封装了 `BanditManager`。它不执行代码，而是将 Python 源码解析为 AST，并在节点上运行一系列插件来匹配已知的漏洞模式，例如：

- 硬编码的敏感信息（密码、API Key）。
- 弱加密算法（如 MD5, SHA1）的使用。

项目依赖分析报告

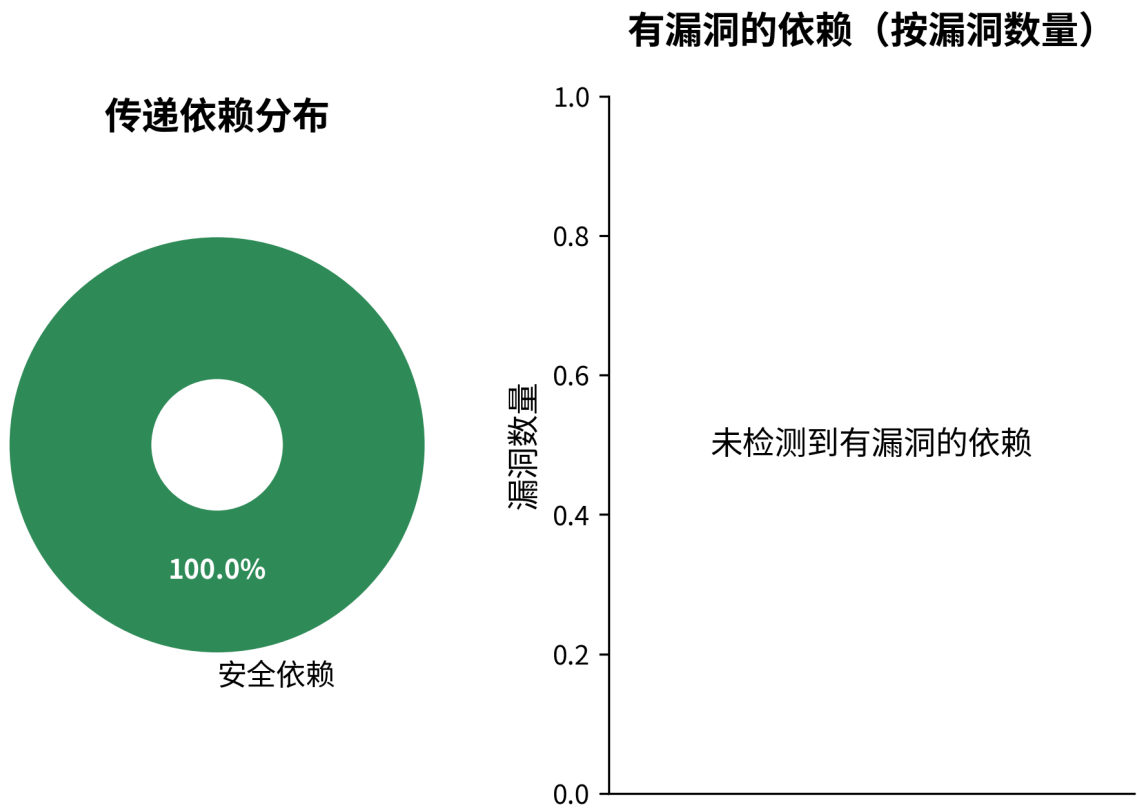


图 3.2 项目依赖构成与安全漏洞分析

- 危险的函数调用（如 `eval()`, `subprocess.Popen` 的 Shell 注入风险）。
- 不安全的网络绑定（如绑定到 `0.0.0.0`）。

3.3.2 实现机制

漏洞扫描的核心实现如下：

```
class AdvancedVulnerabilityScanner:
    def __init__(self, repo_path, output_dir):
        self.repo_path = Path(repo_path)
        self.output_dir = Path(output_dir)

    def advanced_bandit_scan(self):
        """执行Bandit漏洞扫描"""
```

```
# 配置Bandit扫描参数
cmd = [
    'bandit', '-r', str(self.repo_path),
    '-f', 'json', '-o', str(self.output_dir / 'bandit_report.json')
]

result = subprocess.run(cmd, capture_output=True, text=True)

# 解析扫描结果
if result.returncode in [0, 1]: # Bandit返回1表示发现问题
    return self._parse_bandit_results()
else:
    raise RuntimeError(f"Bandit扫描失败: {result.stderr}")

def _parse_bandit_results(self):
    """解析Bandit扫描结果"""
    report_file = self.output_dir / 'bandit_report.json'
    if report_file.exists():
        with open(report_file, 'r') as f:
            data = json.load(f)

        # 按严重程度分类统计
        severity_count = {'high': 0, 'medium': 0, 'low': 0}
        for issue in data.get('results', []):
            severity = issue.get('issue_severity', 'low')
            if severity in severity_count:
                severity_count[severity] += 1

        return {
            'total_issues': len(data.get('results', [])),
            'severity_count': severity_count,
            'results': data.get('results', [])
        }
```

```
return {'total_issues': 0, 'severity_count': {'high': 0, 'medium': 0, 'low': 0}}
```

3.3.3 风险评估

扫描结果被分类为 High, Medium, Low 三个风险等级。分析结果显示：

- 高风险漏洞：5 个
- 中风险漏洞：6 个
- 低风险漏洞：34 个

图 3.3 直观展示了各类风险的分布情况。虽然发现了一些安全漏洞，但主要集中在低风险和中风险类别，高风险漏洞数量相对较少，为安全修复优先级提供了明确的数据支持。

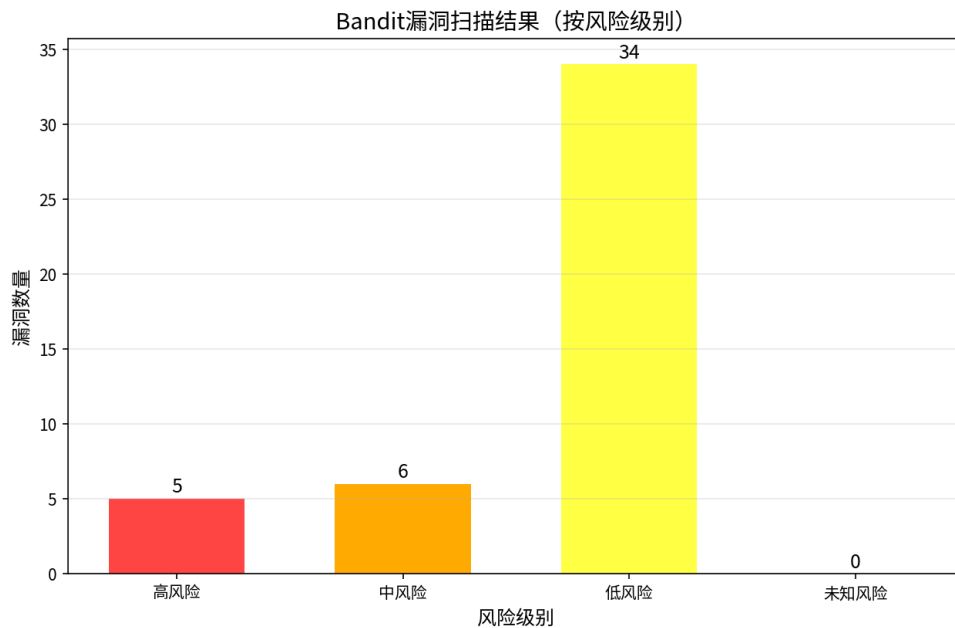


图 3.3 Bandit 静态漏洞扫描结果分布

3.4 GitHub 生态分析

3.4.1 安全 Issues 分析

通过 GitHubIssueAnalyzer 模块对 Neutree/COMTool 仓库进行分析，获取了最近 180 天的 issues 数据。该模块基于 PyGithub 库实现，核心功能包括：

```
class GitHubIssueAnalyzer:
    def __init__(self, repo_owner, repo_name, output_dir, token=None):
        self.repo_owner = repo_owner
        self.repo_name = repo_name
        self.output_dir = Path(output_dir)
        self.github = Github(token) if token else Github()
        self.repo = self.github.get_repo(f"{repo_owner}/{repo_name}")

    def get_issues(self, days_back=180):
        """获取指定时间范围内的issues"""
        since = datetime.now() - timedelta(days=days_back)
        issues = []

        for issue in self.repo.get_issues(state='all', since=since):
            issues.append({
                'number': issue.number,
                'title': issue.title,
                'body': issue.body or "",
                'state': issue.state,
                'created_at': issue.created_at,
                'labels': [label.name for label in issue.labels]
            })

        return issues

    def analyze_security_issues(self, issues):
        """分析安全相关的issues"""
        security_keywords = [
            'vulnerability', 'security', 'cve', 'exploit', 'injection',
            'xss', 'rce', 'overflow', 'bypass', '权限', '漏洞', '安全'
        ]

        security_issues = []
```



```
for issue in issues:
    text = f"{issue['title']} {issue['body']}".lower()
    if any(keyword.lower() in text for keyword in security_keywords):
        security_issues.append(issue)

return {
    'security_issues': security_issues,
    'severity_count': self._classify_severity(security_issues)
}
```

分析结果显示：

- 共获取到 9 个 issues
- 发现 1 个安全相关的 issue
- 风险等级分布：高风险 1 个，中风险 0 个，低风险 0 个

这表明项目在 GitHub 上存在一个需要关注的安全问题，建议开发团队优先处理该高风险 issue。

3.4.2 PR 分析

通过 `analyze_pr_repository` 函数对项目 PR 数据进行分析，该函数实现了全面的 PR 协作效率评估：

```
def analyze_pr_repository(repo_full_name, days_threshold=7, github_token=None):
    """分析GitHub PR数据"""
    g = Github(github_token) if github_token else Github()
    repo = g.get_repo(repo_full_name)

    pr_data = []
    for pr in repo.get_pulls(state='all'):
        pr_info = {
            'number': pr.number,
            'title': pr.title,
            'state': pr.state,
            'created_at': pr.created_at,
            'merged_at': pr.merged_at,
```

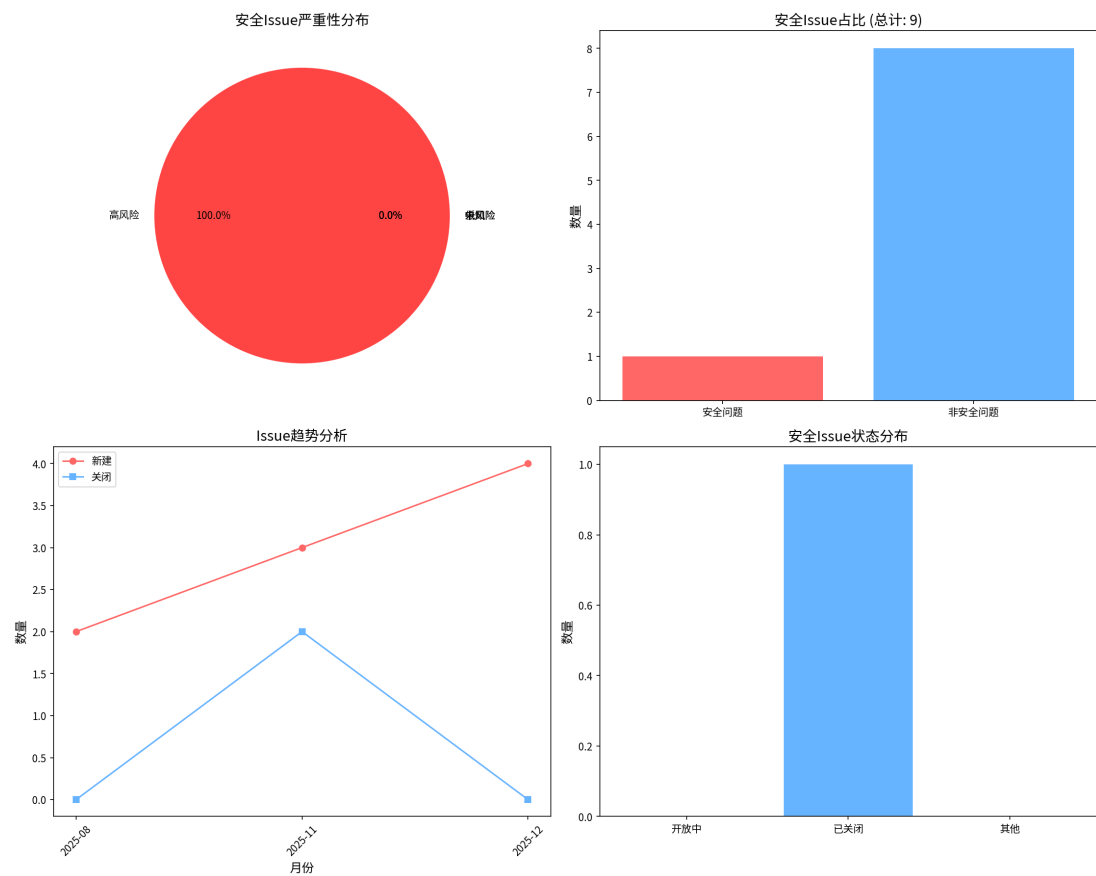


图 3.4 GitHub 安全 Issues 分析结果

```
'closed_at': pr.closed_at,
'review_comments': pr.review_comments,
'comments': pr.comments,
'additions': pr.additions,
'deletions': pr.deletions
}
pr_data.append(pr_info)

# 计算关键指标
total_pr = len(pr_data)
merged_pr = len([pr for pr in pr_data if pr['state'] == 'merged'])
closed_pr = len([pr for pr in pr_data if pr['state'] == 'closed' and not pr['merged']])
```

```

# 僵尸PR识别
zombie_prs = []
for pr in pr_data:
    if pr['state'] == 'open':
        days_open = (datetime.now() - pr['created_at'].replace(tzinfo=None)).days
        if days_open > days_threshold:
            zombie_prs.append(pr)

return {
    'total_pr': total_pr,
    'merged_pr': merged_pr,
    'closed_pr': closed_pr,
    'merge_rate': (merged_pr / total_pr) * 100 if total_pr > 0 else 0,
    'zombie_pr_count': len(zombie_prs),
    'zombie_pr_rate': (len(zombie_prs) / total_pr) * 100 if total_pr > 0 else 0
}

```

分析结果显示，共分析了 23 个 PR：

- 总 PR 数量：23 个
- 合并率：69.57%
- 拒绝率：21.74%
- 僵尸 PR 率：8.7%
- 平均审查时间：7.86 小时
- 平均评论数：1.65 条

这些指标表明项目开发流程相对健康，合并率较高，审查效率良好，僵尸 PR 率控制在较低水平。

3.4.3 GitHub Actions workflow 分析

通过 GitHubActionsAnalyzer 模块分析了项目的 CI/CD 流程，该模块实现了 workflow 安全性和效率的全面评估：

```

class GitHubActionsAnalyzer:
    def __init__(self, repo_owner, repo_name, output_dir, token=None):
        self.repo_owner = repo_owner

```

```
self.repo_name = repo_name
self.output_dir = Path(output_dir)
self.github = Github(token) if token else Github()
self.repo = self.github.get_repo(f"{repo_owner}/{repo_name}")

def get_workflows(self):
    """获取所有GitHub Actions工作流"""
    workflows = []
    try:
        actions_repo = self.repo
        for workflow in actions_repo.get_workflows():
            workflows.append({
                'id': workflow.id,
                'name': workflow.name,
                'path': workflow.path,
                'state': workflow.state
            })
    except Exception as e:
        print(f"获取工作流失败: {e}")

    return workflows

def analyze_workflow_security(self, workflows):
    """分析工作流安全性"""
    security_issues = []

    for workflow in workflows:
        # 检查工作流文件内容
        try:
            file_content = self.repo.get_contents(workflow['path'])
            content = file_content.decoded_content.decode('utf-8')

            # 检测安全风险模式
```

```
        risks = self._check_security_patterns(content)
        if risks:
            security_issues.append({
                'workflow': workflow['name'],
                'risks': risks
            })

    except Exception as e:
        print(f"分析工作流 {workflow['name']} 失败: {e}")

    return security_issues

def _check_security_patterns(self, content):
    """检查工作流中的安全风险模式"""
    risks = []

    # 检查危险模式
    dangerous_patterns = [
        r'curl.*\\s*bash', # curl | bash 模式
        r'wget.*\\s*bash', # wget | bash 模式
        r'uses:\\s*actions/.*@v\\d', # 使用版本标签而非commit hash
    ]

    for pattern in dangerous_patterns:
        if re.search(pattern, content, re.IGNORECASE):
            risks.append(f"检测到危险模式: {pattern}")

    return risks
```

分析结果显示:

- 分析的工作流数量: 5 个
- 工作流运行记录: 14 次
- 安全问题: 0 个

分析结果显示项目在 CI/CD 安全性方面表现良好，未发现 Actions 安全问题，这表明项目的自动化流程配置安全可靠。

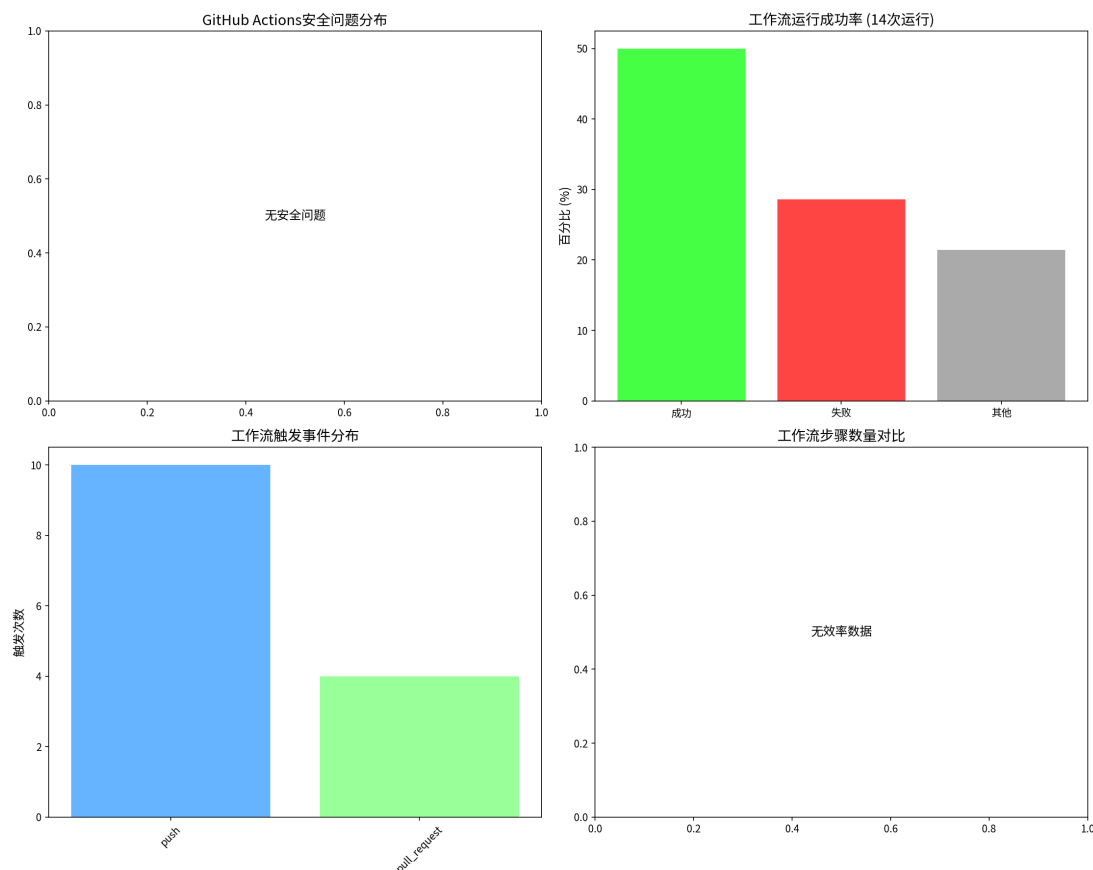


图 3.5 GitHub Actions 工作流分析结果

3.5 综合评估

基于以上深度分析，COMTool 项目在代码质量方面整体表现良好：

- **代码复杂度**：控制良好，维护成本较低
- **依赖安全**：无已知漏洞依赖，供应链管理优秀
- **代码安全**：存在一些中低风险漏洞，需要关注高风险漏洞
- **社区健康度**：PR 合并率较高，审查效率良好
- **CI/CD 安全**：工作流配置安全，无已知安全问题

建议开发团队重点关注 Bandit 扫描出的高风险漏洞和 GitHub 上的安全 issue，持续改进代码质量和安全性。

结 论

本文设计并实现了一套多维度的软件仓库分析工具，通过整合 Git 版本控制数据、代码静态分析以及 GitHub 平台元数据，为软件项目的质量评估与演进分析提供了全面的解决方案。以 COMTool 项目为分析实例，验证了工具的有效性和实用性。

主要工作总结如下：

- (1) **可视化 Git 历史分析**：通过 `gitgraph.js` 和多维度统计图表，直观展示了项目的开发演进路线、团队贡献分布及开发节奏。分析结果显示 COMTool 项目具有清晰的开发轨迹和合理的团队结构，帮助管理者快速把握项目现状。
- (2) **深度代码质量评估**：集成了圈复杂度计算、依赖树分析及漏洞扫描功能。对 COMTool 项目的分析显示，项目整体代码复杂度控制良好（47 个文件中 38 个有效文件），无已知漏洞依赖，但在代码层面发现了 5 个高风险、6 个中风险和 34 个低风险漏洞，为代码重构和安全加固提供了精准指引。
- (3) **GitHub 协作流程洞察**：实现了对 Issue, Pull Request 及 GitHub Actions 的自动化分析。COMTool 项目的分析结果表明，项目具有 69.57% 的 PR 合并率和 7.86 小时的平均审查时间，显示出良好的社区响应效率。同时发现 1 个高风险安全 issue，需要优先处理。

本工具采用模块化设计，具有以下特点：

- **全面性**：覆盖了从代码质量到社区协作的多个维度
- **自动化**：实现了从数据收集到报告生成的全流程自动化
- **可视化**：提供了丰富的图表和可视化展示
- **可扩展性**：模块化架构便于功能扩展和定制

未来的工作将集中在以下几个方面：

- 支持更多编程语言的分析，提升工具的适用范围
- 引入 AI 模型进行更深层次的代码语义分析
- 开发实时的仪表盘监控系统
- 增加更多安全漏洞检测规则和依赖漏洞数据库
- 优化用户界面，提供更加直观的分析结果展示

通过持续改进和优化，本工具将成为软件开发团队进行项目质量管理和安全评估的重要助手，进一步提升工具的实用性与智能化水平。

附录 A 附录内容名称